# `build-pulse`: A Research-Focused Jenkins CI/CD Static Build Analyzer

Ethan Wong [1,2] Hui Zhou [2]

[1]University of Illinois at Chicago [2]Argonne National Laboratory

## Motivation

Complex software projects commonly rely upon continuous-integration and continuous-deployment (CI/CD) platforms to configure extensive build pipelines which streamline development. A single pipeline often holds a substantial number of jobs, each packaging and testing a unique configuration permutation for correctness.

Introducing even a single regression has the ability to propagate failures across an entire pipeline. These regressions cause maintainers to waste significant time finding often trivial underlying issues. `build-pulse`[5] mitigates the impact of these regressions by exhaustively analyzing build pipelines defined in Jenkins, a popular CI/CD platform, coalescing build failures by their underlying issues, and prioritizing regressions by their severity, eliminating manual debugging for maintainers.

## Program Flow

### Stages

1. Artifacts from the runs of each job build configuration are pulled from the Jenkins API and cached in a database to be parsed
2. Unparsed artifacts are pattern matched for snippets which are tagged and stored as issues for analysis
3. Issues are processed for similarity, prioritized, and aggregated as statistics tailored to maintainers' needs
4. A static HTML report is templated with `maud` and outputted for simple viewing in email, webhooks, and browsers

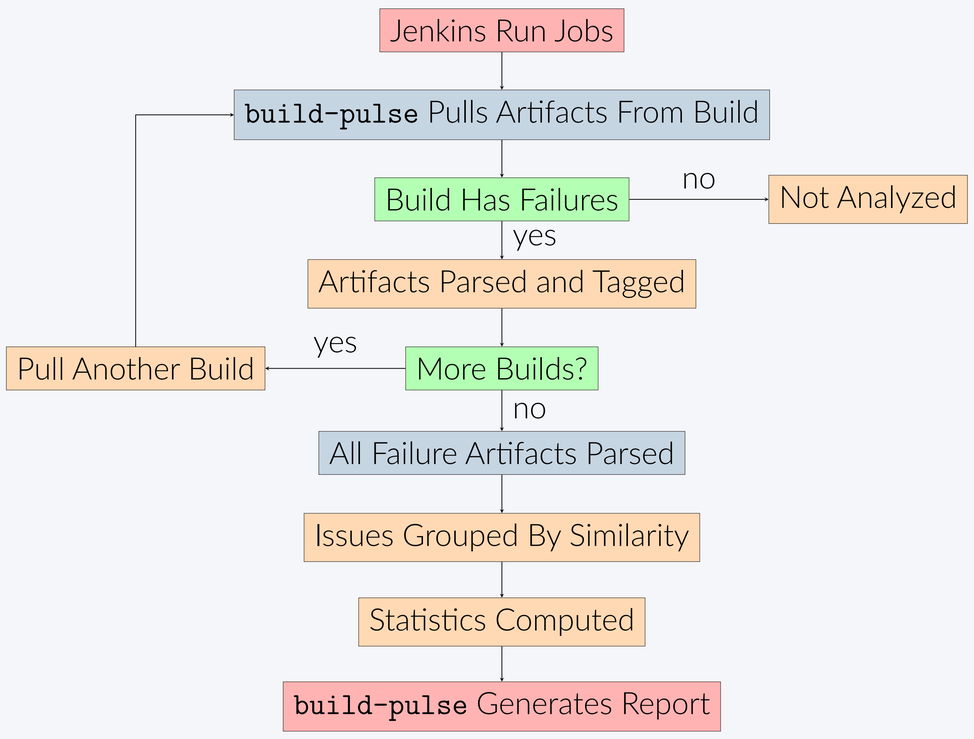



Figure 1. Stage Diagram of `build-pulse`

### Optimization

`build-pulse` is aggressively optimized to maximize performance and maintainer productivity.

- Written in Rust with heavy utilization of `rayon`, a data-parallelism crate akin to OpenMP
- API calls are minimized using the Jenkins Tree Query API[3]
- All runs and their corresponding artifacts are only retrieved once and cached

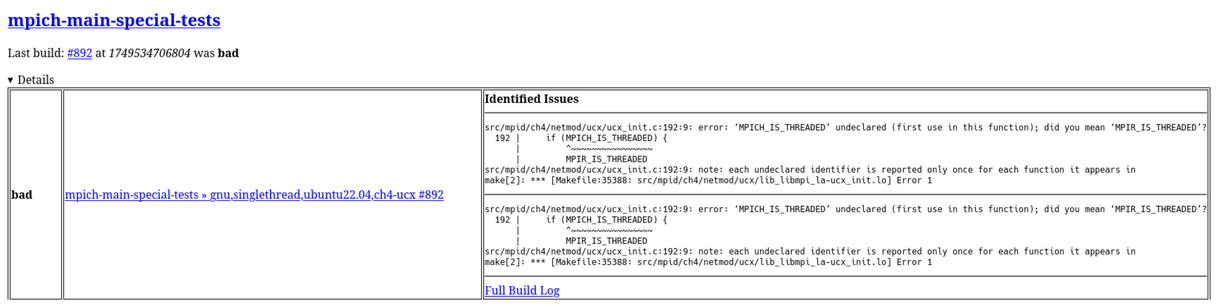
## Issue Tagging

### Tagging

`build-pulse` identifies issues and tags them using regular expressions. Tag categories represent the primary metric aggregated on reports. Tags are defined generically in-order to match issues as *broadly as possible* and allow for parsing to work on a wide variety of tooling.

### Reports

Identified issues are rendered in reports with their snippet and corresponding run information.



**mpich-main-special-tests**

Last build: #892 at *1749534706804* was **bad**

▾ Details

| | | Identified Issues |
|---|---|---|
| **bad** | mpich-main-special-tests » gnu,singlethread,ubuntu22.04,ch4-ucx #892 | `src/mpid/ch4/netmod/ucx/ucx_init.c:192:9: error: 'MPICH_IS_THREADED' undeclared (first use in this function); did you mean 'MPIR_IS_THREADED'?`<br>`192 \| if (MPICH_IS_THREADED) {`<br>`\| ^~~~~~~~~~~~~~~~`<br>`\| MPIR_IS_THREADED`<br>`src/mpid/ch4/netmod/ucx/ucx_init.c:192:9: note: each undeclared identifier is reported only once for each function it appears in`<br>`make[2]: *** [Makefile:35388: src/mpid/ch4/netmod/ucx/lib_libmpi_la-ucx_init.lo] Error 1`<br><br>`src/mpid/ch4/netmod/ucx/ucx_init.c:192:9: error: 'MPICH_IS_THREADED' undeclared (first use in this function); did you mean 'MPIR_IS_THREADED'?`<br>`192 \| if (MPICH_IS_THREADED) {`<br>`\| ^~~~~~~~~~~~~~~~`<br>`\| MPIR_IS_THREADED`<br>`src/mpid/ch4/netmod/ucx/ucx_init.c:192:9: note: each undeclared identifier is reported only once for each function it appears in`<br>`make[2]: *** [Makefile:35388: src/mpid/ch4/netmod/ucx/lib_libmpi_la-ucx_init.lo] Error 1`<br><br>Full Build Log |

Figure 2. Example Report Showing an Identified `cc_emit`

## Defining Similarity

Issues are conservatively grouped and considered "similar" if the normalized Levenshtein distance of their snippets meet a fixed threshold.

### Recursive Definition

The Levenshtein distance metric[1] considers the difference between two snippets $a, b$ as the minimum number of edits needed for $a \to b$.

$$\operatorname{lev}(a,b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \operatorname{lev}\big(\operatorname{tail}(a), \operatorname{tail}(b)\big) & \text{if } \operatorname{head}(a) = \operatorname{head}(b), \\ 1 + \min \begin{cases} \operatorname{lev}\big(\operatorname{tail}(a), b\big) \\ \operatorname{lev}\big(a, \operatorname{tail}(b)\big) \\ \operatorname{lev}\big(\operatorname{tail}(a), \operatorname{tail}(b)\big) \end{cases} & \text{otherwise} \end{cases}$$

### Implementation

In `build-pulse`, the metric is iteratively computed[1] by filling an $|a| * |b|$ matrix until $lev(a,b)$ is solved. A further optimization was made to compute in-place by following a sliding window represented as a single `Vec<T>`, preserving only the last two matrix rows required for computation.

### Normalization

For comparison against snippets of arbitrary length, the Levenshtein distance metric is normalized using an exponential decay model proposed by Lopresti and Zhou[2].

$normalised_lev(a,b) = 1/e^{\alpha lev(a,b)/(m - lev(a,b))}$ where

- $\alpha = 1$ for $normalised_lev(a,b) \in [0.0, 1.0]$
- $m = max(|a|, |b|)$

## Evaluation

`build-pulse` has been actively deployed against MPICH, one of the most popular free-and-open-source MPI implementations. Since its deployment, over **400** unique builds have been parsed nightly.

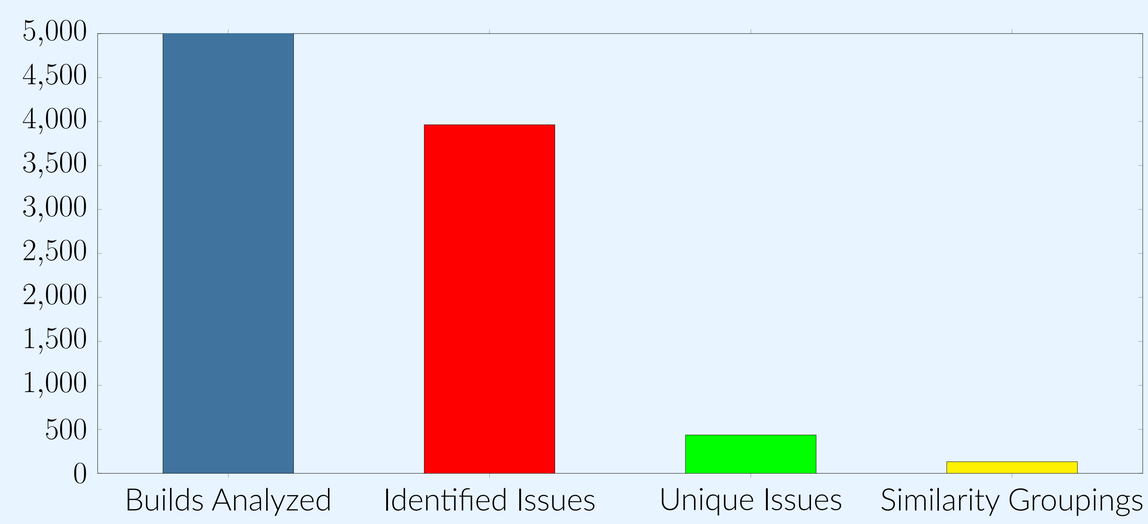



Figure 3. Aggregated Information Since Deployment[6][a]

MPICH maintainers have utilized `build-pulse` reports in email and on Jenkins to stay informed on current project health, changing morning workflows from debugging CI output to *submitting Pull Requests*[4].

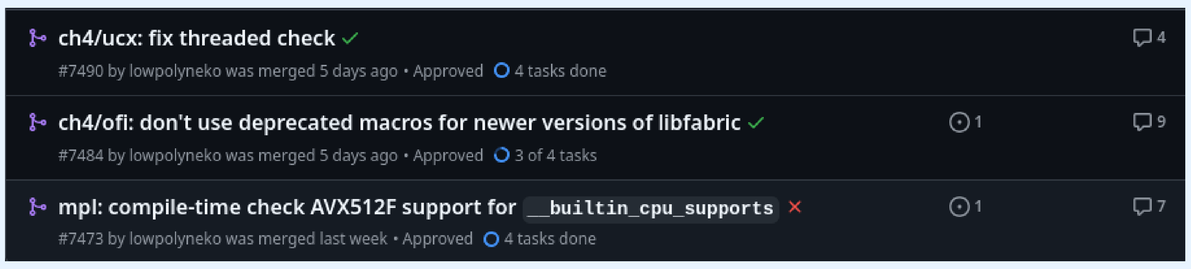



Figure 4. PR Submissions to MPICH Fixing `build-pulse` Issues

## References

[1] Wikipedia contributors. Levenshtein distance. , 2025.

[2] D. Lopresti and J. Zhou. Retrieval strategies for noisy text. Technical report, Matsushita Information Technology Laboratory, 1996.

[3] M. Neale. Taming the jenkins json api with depth and "tree". *CloudBees Blog*.

[4] PMRS. mpich. https://github.com/pmodels/mpich/issues, 2025.

[5] E. Wong. build-pulse. https://github.com/lowpolyneko/build-pulse, 2025.

[6] E. Wong. build-pulse [jenkins]. https://jenkins-pmrs.cels.anl.gov/job/build-pulse, 2025.

U.S. DEPARTMENT of ENERGY

Argonne National Laboratory is a U.S. Department of Energy laboratory managed by UChicago Argonne, LLC.

Argonne NATIONAL LABORATORY